

Rest API

What it is

REST stands for **Representational State Transfer**, and **API** stands for **Application Programming Interface**.

So, a **REST API** is a way for **two systems to communicate over the internet using HTTP**, where resources (like users, products, orders, etc.) are exposed via **URLs**, and you perform operations using **standard HTTP methods** like:

- GET → Retrieve data
- POST → Create new data
- PUT → Update existing data
- DELETE → Remove data

In **stateless communication**, each client request to the server must contain **all the information** needed to understand and process the request.

The server does not remember anything about the previous requests.

Why Is It Used?

-  **Scalability**: Servers can handle thousands of requests as they don't have to remember users.
-  **Simplicity**: No need to maintain user session data on the server.
-  **Resilience**: If one server fails, another can take over easily.

Example of Stateless Communication

1. A user logs in to Flipkart and gets a **JWT token**.
2. Every request to view the cart or place order contains this **token**:

sql

CopyEdit

GET /cart

Authorization: Bearer <JWT token>

3. The server doesn't store who is logged in — it reads the token and identifies the user.

Stateless Example: Visiting a Library

Imagine every time you visit a library:

- You **introduce yourself**.
- Show your **ID card**.

- Mention which **book you want**.
- They process your request **without remembering your previous visit**.

That's a **stateless interaction** — every visit is treated as new.

Stateful Example: Checking into a Hotel

You check into a hotel:

- The receptionist **remembers you**.
- You get a room key.
- You don't have to give your ID every time.

That's a **stateful interaction** — your session is stored.

In Web Terms

Stateful System (like traditional web apps):

- Server stores login status, cart items, session data.
- Requires **session IDs** or cookies.
- Not scalable across servers.

Stateless System (REST API):

- Every request is **self-contained**.
- No session data stored on server.
- Works well in **distributed and scalable systems** like cloud.

Difference between controller and rest controller

@Controller Used in Spring MVC to build **web applications** that return **views (HTML pages)**.

@RestController Introduced in Spring 4.0; used to build **RESTful APIs** that return **JSON or XML** responses directly to the client.

Key Differences between @Controller and @RestController

Feature	@Controller	@RestController
Purpose	Web applications (HTML views)	REST APIs (JSON/XML responses)
Return Type	Returns view names (JSP/HTML/Thymeleaf)	Returns data (JSON/XML)

Feature	@Controller	@RestController
Annotation Composition	Regular annotation	Combines @Controller + @ResponseBody
Requires @ResponseBody?	Yes, if returning data	No, auto-handled
Suitable For	MVC apps with UI	Web services, APIs
Example Response	Web page (e.g., "home.html")	JSON (e.g., {"id":1,"name":"Book"})

@Controller

```
public class WebController {
```

```
    @GetMapping("/home")
```

```
    public String showHomePage(Model model) {
```

```
        model.addAttribute("msg", "Welcome!");
```

```
        return "home"; // Returns home.html or home.jsp
```

```
    }
```

```
}
```

@RestController

```
public class ApiController {
```

```
    @GetMapping("/api/message")
```

```
    public String getMessage() {
```

```
        return "Welcome to the API!";
```

```
    }
```

```
}
```

Building a Simple REST Controller

@RestController

```
@RequestMapping("/api/products")
```

```
public class ProductController {
```

```
    // Your methods will go here
```

```
}
```

@RestController = @Controller + @ResponseBody (auto-converts return values to JSON).

@RequestMapping defines a base path for the controller.

GET

@GetMapping

```
public List<String> getAllProducts() {  
    return List.of("Laptop", "Phone", "Tablet");  
}
```

POST

@PostMapping

```
public String addProduct(@RequestBody String product) {  
    return "Product added: " + product;  
}
```

PUT

@PutMapping("/{id}")

```
public String updateProduct(@PathVariable int id, @RequestBody String product) {  
    return "Product " + id + " updated to: " + product;  
}
```

DELETE

@DeleteMapping("/{id}")

```
public String deleteProduct(@PathVariable int id) {  
    return "Product with ID " + id + " deleted";  
}
```

```
public class Product {
```

```
    private int id;  
    private String name;
```

```
    // constructor, getters, setters
```

```
}
```

@GetMapping("/{id}")

```

public Product getProduct(@PathVariable int id) {
    return new Product(id, "Laptop");
}

```

@PathVariable

```

@GetMapping("/{category}/{id}")
public String getProductByCategory(
    @PathVariable String category,
    @PathVariable int id) {
    return "Category: " + category + ", Product ID: " + id;
}

```

@RequestParam

```

@GetMapping("/search")
public String searchProducts(@RequestParam String name, @RequestParam int limit) {
    return "Searching for: " + name + ", Limit: " + limit;
}

```

📌 URL:

GET /api/products/search?name=Phone&limit=5

Searching for: Phone, Limit: 5

Complete Example

```

@RestController
@RequestMapping("/api/foods")
public class FoodController {

    @GetMapping
    public List<String> getAllFoods() {
        return List.of("Pizza", "Burger", "Pasta");
    }

    @GetMapping("/{id}")
    public String getFoodById(@PathVariable int id) {
        return "Food with ID: " + id;
    }
}

```

```
}
```

```
@PostMapping
```

```
public String addFood(@RequestBody String foodName) {  
    return "Food added: " + foodName;  
}
```

```
@PutMapping("/{id}")
```

```
public String updateFood(@PathVariable int id, @RequestBody String updatedFood) {  
    return "Food with ID " + id + " updated to: " + updatedFood;  
}
```

```
@DeleteMapping("/{id}")
```

```
public String deleteFood(@PathVariable int id) {  
    return "Deleted food with ID: " + id;  
}
```

```
@GetMapping("/search")
```

```
public String searchFood(@RequestParam String name) {  
    return "Searching for food: " + name;  
}  
}
```

Request and Response Handling

- ❑ Use `@PathVariable` to extract dynamic values from the URL.
- ❑ Use `@RequestParam` for optional query parameters.
- ❑ Use `@RequestBody` for receiving JSON input.
- ❑ Use `ResponseEntity` to customize response status and headers.
- ❑ Use `@ControllerAdvice` and `@ExceptionHandler` for global and local error handling.

```
@GetMapping("/users/{id}")
```

```
public String getUserById(@PathVariable int id) {
```

```
    return "User ID: " + id;
}
```

Url: GET /users/10

Output: User ID: 10

```
@GetMapping("/search")
public String searchUser(@RequestParam String name) {
    return "Searching user: " + name;
}
```

Input: GET /search?name=John

Output: Searching user: John

@RequestBody – Accepting JSON Data

@RequestBody is used when JSON data is sent in the body of the request. It's automatically converted to a Java object using Jackson.

```
@PostMapping("/products")
public String addProduct(@RequestBody Product product) {
    return "Added product: " + product.getName();
}
```

Input Request (POST /products):

```
{
  "id": 101,
  "name": "Laptop"
}
```

 Output:

Added product: Laptop

JSON request is converted to a Product object. Spring automatically binds fields based on matching names.

Handling Form Data using @ModelAttribute

@ModelAttribute binds form inputs to a Java object (used in traditional web forms).

```
@PostMapping("/register")
```

```
public String registerUser(@ModelAttribute User user) {
    return "Registered: " + user.getName();
}
```

```
<form action="/register" method="post">
    <input name="name" value="Alice"/>
    <input name="email" value="alice@mail.com"/>
    <button type="submit">Register</button>
</form>
```

Registered: Alice

Form fields (name, email) are mapped directly to the User object properties.

Custom Response with ResponseEntity

ResponseEntity gives full control over:

- **Body**
- **HTTP status code**
- **Headers**

```
@PostMapping("/save")
```

```
public ResponseEntity<String> saveData(@RequestBody String data) {
    return new ResponseEntity<>("Saved: " + data, HttpStatus.CREATED);
}
```

url: POST /save

Body: "Hello"

Output: HTTP/1.1 201 Created

Saved: Hello

You explicitly set the status to 201 Created, useful when resource creation is involved.

Custom Headers in Response

Add custom headers using HttpHeaders for metadata (e.g., app version).

```
@GetMapping("/info")
```

```
public ResponseEntity<String> getInfo() {
    HttpHeaders headers = new HttpHeaders();
    headers.add("X-App-Version", "1.0");
}
```



```
    return new ResponseEntity<>("App Info", headers, HttpStatus.OK);  
}
```

url: GET /info

output: HTTP/1.1 200 OK

X-App-Version: 1.0

Body: "App Info"

Exception Handling – Global using @ControllerAdvice

Handle errors in a **central place** using @ControllerAdvice.

```
@GetMapping("/users/{id}")
```

```
public String getUser(@PathVariable int id) {  
    if (id < 0) throw new IllegalArgumentException("Invalid ID");  
    return "User ID: " + id;  
}
```

```
@ControllerAdvice
```

```
public class GlobalErrorHandler {
```

```
    @ExceptionHandler(IllegalArgumentException.class)
```

```
    public ResponseEntity<String> handleInvalidId(IllegalArgumentException ex) {
```

```
        return new ResponseEntity<>("Error: " + ex.getMessage(), HttpStatus.BAD_REQUEST);
```

```
    }
```

```
}
```

Input:

GET /users/-1



Output:

HTTP/1.1 400 Bad Request

Error: Invalid ID

The @ControllerAdvice intercepts exceptions and converts them into **friendly HTTP responses**.

RESTful Resource Representation with DTOs

What is a DTO?

A **DTO (Data Transfer Object)** is a simple Java class that contains **only the data you want to expose to the outside world** (usually through a REST API).

Why Not Return Entity Directly?

Problem with Returning Entity	Solution via DTO
Exposes sensitive fields (e.g., password)	DTO shows only needed data
Tight coupling with DB structure	DTO is independent
Cannot customize response easily	DTO is flexible
Harder to handle versioning	DTOs can be versioned separately

@Entity

```
public class User {  
    private int id;  
    private String username;  
    private String password;  
    private String email;  
    private LocalDate dob;  
}
```

✗ If you return this entity, it may leak password or dob.

Create a Safe DTO for API Response

```
public class UserDTO {  
    private String username;  
    private String email;  
}
```

- ☐ Only return username and email in API response.
- ☐ Keep internal fields (id, password, dob) hidden.

Manual Mapping: Entity → DTO

```
public class UserMapper {  
    public static UserDTO convertToDto(User user) {  
        UserDTO dto = new UserDTO();
```

```

        dto.setUsername(user.getUsername());

        dto.setEmail(user.getEmail());

        return dto;
    }
}

Controller Code

@RestController
@RequestMapping("/api/users")

public class UserController {

    @GetMapping("/{id}")

    public UserDTO getUser(@PathVariable int id) {

        User user = userService.findById(id);

        return UserMapper.convertToDto(user);

    }
}

```

Input:

GET /api/users/1

Output:

```

{
  "username": "john_doe",
  "email": "john@gmail.com"
}

```

Mapping with ModelMapper (Auto Mapping)

```

<dependency>

  <groupId>org.modelmapper</groupId>

  <artifactId>modelmapper</artifactId>

  <version>3.1.0</version>

</dependency>

@Autowired

```

```
private ModelMapper modelMapper;

@GetMapping("/{id}")
public UserDTO getUser(@PathVariable int id) {
    User user = userService.findById(id);
    return modelMapper.map(user, UserDTO.class);
}
```

Customizing JSON Serialization & Deserialization

Serialization Converting a Java object **into JSON** format (outgoing response)

Deserialization Converting a JSON **into a Java object** (incoming request)

Rename Field Name in JSON

```
@JsonProperty("user_name")
private String username;
```

 JSON Output:

```
{
  "user_name": "john_doe"
}
```

Hide Field from Response

```
@JsonIgnore
private String internalNote;
```

 JSON Output:

```
{
  // field excluded
}
```

Format Dates

```
@JsonFormat(pattern = "yyyy-MM-dd")
private LocalDate dob;
```

 Output:

```
{  
  "dob": "2000-01-15"  
}
```

Conditional Fields

```
@JsonInclude(JsonInclude.Include.NON_NULL)
```

```
private String nickname;
```

 If nickname is null, it will be omitted.

A DTO (Data Transfer Object) is a plain Java class used to **transfer data across layers**, especially in REST APIs. Instead of returning entity objects, we return DTOs to **protect sensitive fields, improve performance**, and **manage API versioning**.

Spring Boot supports mapping using **manual methods** or tools like **ModelMapper**. You can customize how DTOs are serialized/deserialized using Jackson annotations like `@JsonProperty`, `@JsonIgnore`, and `@JsonFormat`.

Versioning can be handled by creating **separate DTO classes (v1, v2)** and using versioned endpoints (e.g., `/v1/users`, `/v2/users`). This is critical in **real-world apps like Amazon, Flipkart, or Swiggy** to avoid breaking older clients.

RESTful CRUD Operations

. What is CRUD?

CRUD stands for:

Operation	Description	HTTP Method
Create	Add new resource	POST
Read	Get existing resource(s)	GET
Update	Modify existing resource	PUT / PATCH
Delete	Remove a resource	DELETE

We'll use a simple **Product Entity** for our example:

```
@Entity
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue
```

```
    private Long id;
```

@NotBlank

private String name;

@Positive

private double price;

private String description;

}

@PostMapping("/products")

public ResponseEntity<Product> createProduct(@Valid @RequestBody Product product) {

Product saved = productRepository.save(product);

return new ResponseEntity<>(saved, HttpStatus.CREATED);

}

Request (Input JSON):

{

"name": "Mobile",

"price": 10000,

"description": "Latest smartphone"

}



Response:

{

"id": 1,

"name": "Mobile",

"price": 10000,

"description": "Latest smartphone"

}



Uses:

- @PostMapping
- @RequestBody
- @Valid → validates fields using annotations

```
@GetMapping("/products")

public List<Product> getAllProducts() {

    return productRepository.findAll();

}
```

 Output:

```
[

  {

    "id": 1,

    "name": "Mobile",

    "price": 10000

  },

  {

    "id": 2,

    "name": "Laptop",

    "price": 55000

  }

]
```

```
@GetMapping("/products/{id}")

public ResponseEntity<Product> getProduct(@PathVariable Long id) {

    return productRepository.findById(id)

        .map(ResponseEntity::ok)

        .orElse(ResponseEntity.notFound().build());

}
```

 Request:

GET /products/1

 Output:

```
{

  "id": 1,

  "name": "Mobile",

  "price": 10000}
```

```
@PutMapping("/products/{id}")
```

```
public ResponseEntity<Product> updateProduct(@PathVariable Long id, @Valid @RequestBody  
Product updatedProduct) {
```

```
    return productRepository.findById(id)  
        .map(product -> {  
            product.setName(updatedProduct.getName());  
            product.setPrice(updatedProduct.getPrice());  
            product.setDescription(updatedProduct.getDescription());  
            productRepository.save(product);  
            return ResponseEntity.ok(product);  
        }).orElse(ResponseEntity.notFound().build());  
}
```

 Input:

```
{  
  "name": "Updated Mobile",  
  "price": 11000,  
  "description": "New specs"  
}
```

 Output:

```
{  
  "id": 1,  
  "name": "Updated Mobile",  
  "price": 11000,  
  "description": "New specs"  
}
```

```
@DeleteMapping("/products/{id}")
```

```
public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
```

```
    if (!productRepository.existsById(id)) {  
        return ResponseEntity.notFound().build();  
    }  
    productRepository.deleteById(id);  
}
```



```
return ResponseEntity.noContent().build();
}
```

 Request:

DELETE /products/1

 Output:

HTTP 204 No Content

Input Validation with Annotations

To ensure **data correctness** before saving to the DB.

Validation in Entity:

```
public class Product {
    @NotBlank(message = "Name is required")
    private String name;

    @Positive(message = "Price must be positive")
    private double price;
}

public ResponseEntity<Product> create(@Valid @RequestBody Product p) { ... }
```

If validation fails, Spring returns:

 Error Output:

```
{
  "errors": [
    "Name is required",
    "Price must be positive"
  ]
}
```

What is Content Negotiation?

Content Negotiation is a mechanism by which the client tells the server **what format (media type)** it wants the response in — such as **JSON**, **XML**, or even **CSV** or **PDF**.

When you open a webpage in a browser or a mobile app:

- The app might say: Accept: application/json

- The browser might say: Accept: text/html

The server reads this and returns the correct format.

How Spring Boot Handles Content Negotiation by Default

Spring Boot supports **automatic content negotiation** using the Accept HTTP header.

Example Entity:

```
public class Product {  
    private Long id;  
    private String name;  
    private double price;  
}  
  
@GetMapping(value = "/product", produces = { "application/json", "application/xml" })  
public Product getProduct() {  
    return new Product(1L, "Laptop", 45000.0);  
}
```

Request from Client:

GET /product

Accept: application/json

Response:

```
{  
  "id": 1,  
  "name": "Laptop",  
  "price": 45000.0  
}
```

Request:

GET /product

Accept: application/xml

Response:

```
<Product>  
  
  <id>1</id>  
  
  <name>Laptop</name>
```

```
<price>45000.0</price>
```

```
</Product>
```

. How to Enable XML Support in Spring Boot

Add XML Dependency:

Spring Boot doesn't include XML by default.

```
<dependency>
```

```
<groupId>com.fasterxml.jackson.dataformat</groupId>
```

```
<artifactId>jackson-dataformat-xml</artifactId>
```

```
</dependency>
```

✓ Now your application can **automatically serialize** to both JSON and XML.

Producing and Consuming Custom Media Types

When do we need this?

- You want a **custom format** for APIs (like `application/vnd.myapp.v1+json`)
- You are building **versioned APIs**
- You want to handle **non-standard types** (CSV, PDF)

Producing Custom Media Type (Versioned JSON)

```
@GetMapping(value = "/product", produces = "application/vnd.myapp.v1+json")
```

```
public Product getProductV1() {
```

```
    return new Product(1L, "Mobile", 9999.0);
```

```
}
```

Request:

GET /product

Accept: application/vnd.myapp.v1+json

Response:

```
{
```

```
  "id": 1,
```

```
  "name": "Mobile",
```

```
  "price": 9999.0
```

```
}
```

You can also use this pattern for:

- **API versioning**

- Returning different formats for different clients

Spring Boot Actuator for REST Monitoring

Spring Boot **Actuator** is a powerful module that provides **production-ready features** like monitoring, metrics, health checks, auditing, environment info, and more.

You can **monitor RESTful services**, **expose custom metrics**, and **secure endpoints** easily.

Feature	Use Case
Health Checks	Is your REST API running?
Metrics	Monitor performance, request count, memory
Info	Show app version, build info
Custom Endpoints	Track business-specific metrics
Security	Protect sensitive monitoring data

Add Dependency

Add this to your pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Enable in application.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

Default Actuator Endpoints

Once actuator is added and enabled, access these:

Endpoint	Description
/actuator/health	Shows app health status
/actuator/info	App info (e.g., version, name)
/actuator/metrics	System metrics (CPU, memory, requests)
/actuator/env	Environment variables
/actuator/beans	Beans in Spring context

Endpoint	Description
/actuator/mappings	All controller routes

Security & Authentication in RESTful APIs using Spring Boot

-  Securing endpoints with Spring Security
-  Authentication & Authorization
-  JWT Token-based Authentication
-  CORS Handling (Cross-Origin Resource Sharing)
-  Real-world examples + input/output + configurations

Add Security Dependency

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-security</artifactId>

</dependency>

Authentication vs Authorization

Concept	Description	Example
Authentication	Who are you?	Login with username & password
Authorization	What can you access?	Admin can delete users, user cannot

JWT (JSON Web Token) Authentication in REST APIs

What is JWT?

JWT is a **stateless** and **compact** token format to authenticate REST APIs.

Header.Payload.Signature

- Encoded in Base64
- No session stored on server
- Used in Authorization header:
Authorization: Bearer <token>

JWT Authentication Workflow

1. User sends login credentials
2. Server validates & returns JWT token
3. Client stores token (usually in localStorage)

4. Client sends token with every API request
5. Server validates JWT & grants/denies access

```
<!-- Spring Security -->
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-security</artifactId>
```

```
</dependency>
```

```
<!-- JWT -->
```

```
<dependency>
```

```
<groupId>io.jsonwebtoken</groupId>
```

```
<artifactId>jjwt</artifactId>
```

```
<version>0.9.1</version>
```

```
</dependency>
```

```
public class UserDTO {
```

```
    private String username;
```

```
    private String password;
```

```
}
```

```
@Component
```

```
public class JwtUtil {
```

```
    private String secret = "my-secret-key";
```

```
    public String generateToken(String username) {
```

```
        return Jwts.builder()
```

```
            .setSubject(username)
```

```
            .setIssuedAt(new Date())
```

```
            .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60)) // 1 hour
```

```
            .signWith(SignatureAlgorithm.HS256, secret)
```

```
            .compact();
```

```
    }
```

```
    public String extractUsername(String token) {
```

```
        return Jwts.parser().setSigningKey(secret)
```

```

        .parseClaimsJws(token)

        .getBody()

        .getSubject();
    }

    public boolean validateToken(String token, String username) {
        return username.equals(extractUsername(token));
    }
}

@RestController

public class AuthController {

    @Autowired private JwtUtil jwtUtil;

    @PostMapping("/login")
    public ResponseEntity<String> login(@RequestBody UserDTO user) {
        if ("admin".equals(user.getUsername()) && "password".equals(user.getPassword())) {
            String token = jwtUtil.generateToken(user.getUsername());
            return ResponseEntity.ok(token);
        }
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid credentials");
    }
}

@Component

public class JwtFilter extends OncePerRequestFilter {

    @Autowired private JwtUtil jwtUtil;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

```

```

String authHeader = request.getHeader("Authorization");

if (authHeader != null && authHeader.startsWith("Bearer ")) {
    String jwt = authHeader.substring(7);
    String username = jwtUtil.extractUsername(jwt);
    if (username != null) {
        UsernamePasswordAuthenticationToken authToken =
            new UsernamePasswordAuthenticationToken(username, null, List.of());
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}

filterChain.doFilter(request, response);
}
}

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired private JwtFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http.csrf().disable()
            .authorizeRequests()
            .antMatchers("/login").permitAll()
            .anyRequest().authenticated()
            .and()
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}

POST /login
Content-Type: application/json

```



```
{  
  "username": "admin",  
  "password": "password"  
}
```

<JWT_TOKEN_STRING>

GET /products

Authorization: Bearer <JWT_TOKEN_STRING>

Handling Cross-Origin Resource Sharing (CORS)

Browsers block AJAX requests from other domains (for security). CORS allows **cross-origin** requests.

@Configuration

```
public class CorsConfig implements WebMvcConfigurer {  
    @Override  
    public void addCorsMappings(CorsRegistry registry) {  
        registry.addMapping("/*")  
            .allowedOrigins("http://localhost:3000") // frontend origin  
            .allowedMethods("GET", "POST", "PUT", "DELETE")  
            .allowedHeaders("*")  
            .allowCredentials(true);  
    }  
}
```

Best Practices for Testing REST APIs

Best Practice	Reason
Isolate layers using @MockBean	Unit test only one layer
Use MockMvc	Avoid full server startup
Use test profile with H2 DB	Clean, fast, repeatable tests
Use @BeforeEach to reset data	Ensure consistent test results
Use descriptive method names	Helps identify test failures
Cover positive and negative cases	Real-world robustness

